

G06 · TNT

G06 · TNT — 학습을 짜게만들기

이 권은 앞의 G03~G05(Titans·Miras·Atlas)가 만든 “읽으며학습하는 메모리”를 어떻게 빠르고 짜게 훈련하느냐의 이야기다. 앞권들이 “메모리가 무엇을 배워야 하는가” (update rule의 내용)를 채웠다면, TNT는 그 메모리들이 GPU/TPU에서 하드웨어를 놀리지 않고 돌게만드는 방법을 다룬다. 그래서 이 권의 주인공은 새 수식이 아니라 **chunk 크기**라는 하나의 숫자와, 그 숫자가 만드는 딜레마를 두단계로 쪼개는 레시피다.

1. 이 권이 푸는 문제: chunk 크기딜레마

먼저 앞 권의 그림을 한 문장으로 복습하자. 이 계열의 “메모리”는 고정된표가 아니라 **입력을 읽으면서 계속 학습시키는 작은 신경망**이다. 토큰이 하나 들어올 때마다 이 작은 신경망을 조금씩 갱신하고 (write), 질문이 오면 그 신경망에 물어본다 (read).

문제는 “조금씩 갱신”을 **토큰 하나마다** 순서대로 하면 하드웨어가논다는 것이다. GPU/TPU는 큰 행렬 곱셈 하나를 좋아하지, 작은 연산 수천개를 순서대로 처리하는 걸 싫어한다.

비유 택배 트럭을 생각하자. 소포 하나를 싣고 출발, 배달, 돌아와서 다음 소포하나를 싣고 출발 ... 이렇게 하면 트럭 (하드웨어)은 대부분 도로 위에서 텅 빈 채로 달린다. 트럭을 꽉 채워 한 번에 많이 실으면 (=큰 chunk) 왕복이줄어 효율이 오른다. 하지만 소포를 너무 오래 창고에 쌓아두면 (=chunk가커서 갱신이 늦으면) 주소가 바뀐 소포가 생겨 배달 품질이 떨어진다.

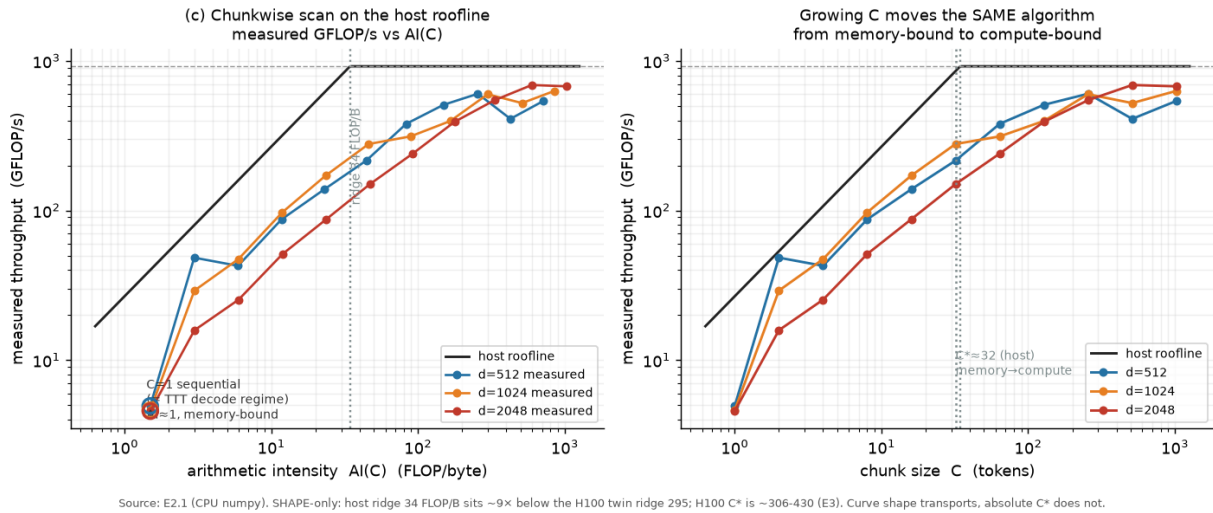
이것이 **chunk 크기 딜레마**다. chunk란 “한 번에 묶어서 병렬로처리하는 토큰 묶음”이다.

- **chunk가 크면:** 큰 행렬 곱 하나로 처리 → 하드웨어 잘 쏘 (빠름). 하지만 묶음 안의 모든 토큰이 **묶음 시작 시점의 낡은 메모리상태**를 기준으로 갱신을 계산하므로 (이걸 stale-snapshot이라 부른다), 근사가 거칠어져 **품질↓**.
- **chunk가 작으면:** 갱신이 자주·신선하게 일어나 **품질↑**. 하지만 작은 연산이 대량으로 쪼개져 하드웨어가 놀아 **느림**.

실무는 chunk를 16~64 정도에 고정해 타협해 왔다. 그 대가가얼마나 큰지 TNT 논문이 인용한 수치가 충격적이다: 이 계열의 좋은 (작은 chunk) 훈련은 하드웨어 최대 성능 (peak FLOPs) 대비 5~10% **미만**밖에 못 쓴다. 나머지 90%는 놀고 있다.

직관 “얼마나 빠른가”와 “얼마나 정확한가”를 **같은 하나의손잡이 (chunk 크기)가 동시에 결정**한다는 게 문제의 핵심이다. 손잡이를한쪽으로 돌리면 속도를 얻고 품질을 잃고, 반대로 돌리면 그 반대다. TNT의전략은 이 손잡이를 **두 개로 쪼개는** 것이다.

이 딜레마를 하드웨어 관점에서 그리면 아래 roofline 그래프가 된다. 가로축은 arithmetic intensity(연산량 ÷ 메모리 접근량, 즉 “한 번 데이터를읽어와서 얼마나 많이 계산하는가”)이고, chunk가 커질수록 오른쪽으로이동해 하드웨어의 지붕 (roof)에 가까워진다.



Chunkwise scan 을 호스트 roofline 위에 얹은 그림. chunk 크기 C 가 커질수록 arithmetic intensity $AI(C)$ 가 오른쪽으로 이동해 실측 GFLOP/s 가지붕 (하드웨어 최대 성능) 에 다가간다 — 작은 chunk 에서는 메모리 대역폭에 묶여 (memory-bound) 하드웨어가 놀고, 큰 chunk 에서만 compute-bound 가 된다. 출처: 본서 자체 실험 (E2.1) — 원저자 그림이 아님.

시스템 모델링 관점 5~10% utilization 의 원인은 FLOPs 가 부족해서가 아니라 **arithmetic intensity** 가 부족해서다. 작은 chunk 는 상태를 읽어와서 몇 번 계산 하지 못하고 다시 쓰는 memory-bound 작업이다. 모델링할 때 chunk 크기 C 는 곧 “작업 단위 하나의 arithmetic intensity” 로 읽으면 된다 — 대략 $AI \propto C$. TNT 의 모든 트릭은 결국 이 C 를 크게하거나 (global), 작은 작업을 많이 · 독립적으로 만들어 batch 축에 쌓아 (local) intensity 를 확보하는 것으로 요약된다.

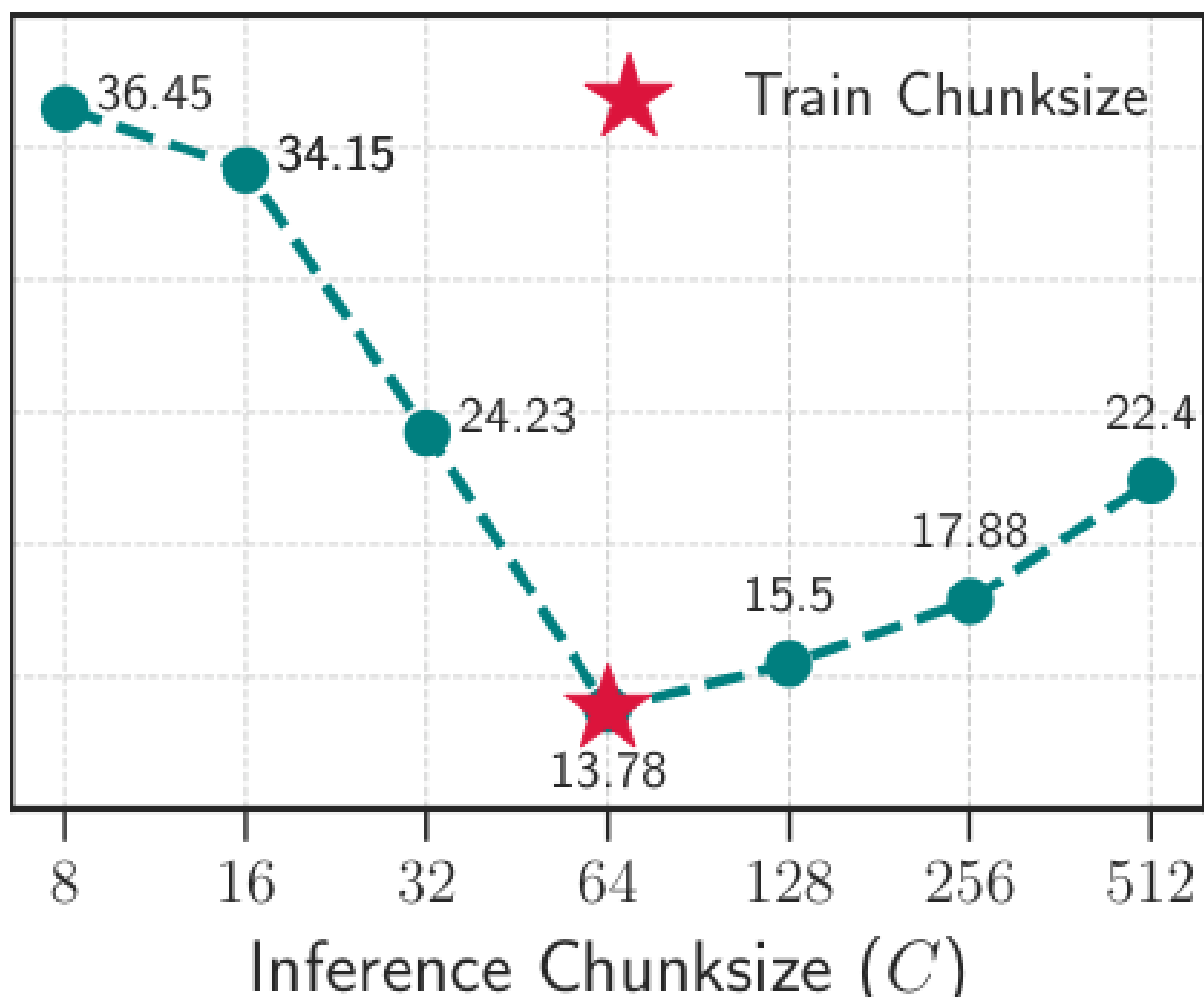
2. 딜레마의 증거: 훈련 chunk 에 갇힌 모델

TNT 는 여기에 새로운 실험적 발견을 하나 더한다. 550M 크기의 Titans 모델을 chunk 크기 $C = 64$ 로 훈련한 뒤, 추론 때만 chunk 크기를 바꿔가며 품질 (perplexity, 낮을수록 좋음) 을 재봤다.

추론 chunk 크기	perplexity (낮을수록 좋음)
8	36.45
16	34.15
32	24.23
64 (훈련값)	13.78 (최적)
128	15.50
256	17.88
512	22.40

훈련 때 쓴 값 (64) 에서만 품질이 최고이고, 양쪽으로 벗어나면 급격히 나빠진다. 특히 왼쪽이 반직관적이다. chunk 를 더 작게 (8) 하면 갱신이 더 신선해지니 직관적으로는 더 좋아야 하는데, 실제로는 품질이 2.6 배 이상 폭발한다.

직관 모델이 훈련 때 본 chunk 해상도에 **과적응 (over-specialize)** 했다. “나는 64 개 씩 묶어서 갱신하는 세계에서만 잘 동작해” 라고 굳어버린 것이다. 이것이 이 계열의 골칫거리인 **chunk-size mismatch** (train/serve 불일치) 다: 같은 체크포인트라도 서빙 때 갱신 주기를 바꾸면 전혀 다른 품질이 나온다.



550M Titans 를 $C=64$ 로 pre-train 한 뒤 추론 chunk 크기만 바뀌가며 validation perplexity. 훈련값 (별표, $C=64$) 에서 13.78로 최적이고 양쪽으로 벗어나면 급격히 악화 — 특히 더 작은 chunk(왼쪽) 가 직관과 반대로 36.45 까지 폭발한다. V자 바닥이 정확히 훈련 chunk 에 걸려 있다. 출처: 저자, TNT(arXiv:2511.07343) Fig.2 — 원저자 그림.

3. TNT의 한 줄 요약

TNT의 이름은 “Titans iNside Titans” (또는 “TTT iNside TTT”)의 약자다 — 메모리 안에 메모리를 중첩한 계층 구조를 가리킨다. 핵심 주장은 이렇다.

핵심 훈련 효율과 추론 품질을 하나의 chunk 크기가 동시에 결정하게 놔두지 말고, 두 단계로 분리하라. - Stage 1: 계층 메모리 (큰 chunk global + 작은 chunk local)로 하드웨어를 꽉 채워 최대 throughput으로 pre-training. - Stage 2: 전체 비용의 약 5~8%만 더 써서 작은 chunk로 짧게 fine-tuning → chunk-1 decode를 품질과 정렬.

결과를 미리 말하면 (150M 모델 기준): 가장 정확한 Titans baseline($C = 8$) 대비 목표 품질 도달까지 최대 17.37배 빠르면서, 품질 (perplexity)은 오히려 더 좋고 (23.09 vs 25.07) 바닐라 Transformer(23.58)도 이긴다.

TNT는 새 아키텍처가 아니라 훈련 방법론이다. 표현력이 아니라 throughput과 chunk 경제학을 판다.

4. 먼저 복습: 메모리는 두 개의 연산이다

TNT는 이 계열이 암묵적으로 쓰던 구조를 딱 두 개의 per-token 연산으로 정리한다. 작은 신경망 $\mathcal{M}$ 의 가중치 W 가 있고, 토큰마다:

$$W_t = W_{t-1} - \eta_t \nabla_W \ell(W_{t-1}; k_t, v_t) \quad (\text{write / Compression})$$

$$y_t = \mathcal{M}(q_t; W_t) \quad (\text{read / Retrieval})$$

기호 풀이 - W_t = fast weights. 시각 t 에서 메모리 신경망의 가중치 (=메모리의 “내용”). 우리가 계속 갱신하는 상태. - $\mathcal{M}(\cdot; W)$ = 메모리 신경망 자체. 입력 벡터를 받아 출력 벡터를 내는 작은 MLP. - k_t, v_t, q_t = 입력 토큰에서 뽑은 key·value·query 벡터 (각 d 차원). 훈련된 projection이 만든다. - ℓ = inner loss. 메모리가 얼마나 틀렸는지 재는 함수. 기본형은 “key를 넣었을 때 value가 나오게” 맞추는 회귀 오차 $\|\mathcal{M}(k; W) - v\|_2^2$. - η_t = 학습된 per-token learning rate. 이번 토큰에서 메모리를 얼마나 세게 갱신할지. 토큰마다 값이 다르다. - $\nabla_W \ell = \ell$ 의 W 에 대한 gradient. “이 방향으로 W 를 바꾸면 덜 틀린다”는 방향.

직관 write는 “이번 토큰에 대해 메모리가 틀린 만큼 (ℓ), 덜 틀리는 방향으로 가중치를 조금 (η_t) 밀어준다”는 뜻이다. surprise(놀람) 가 클수록 gradient가 크고, 그만큼 크게 배운다. read는 “질문 q_t 를 지금까지 학습된 메모리 신경망에 통과시켜 답을 받는다”는 뜻이다.

이 식은 ~라는 뜻이다. 메모리 = 토큰이 지나갈때마다 gradient descent 한 스텝씩 학습되는 작은 모델. 읽기 = 그 모델의 forward pass.

여기서 결정적 사실 하나: 이 갱신은 **비선형**이다. $\nabla_W \ell$ 을 계산하려면 deep 신경망 $\mathcal{M}$ 의 forward와 backward를 통과해야 한다. 이게 뒤에서 볼 모든 어려움의 근원이다 — 상태전이가 비선형이면 GLA/DeltaNet 같은 linear attention의 빠른 chunk 커널을 **그대로** 못 가져온다.

시스템 모델링 관점 여기서 이미 KV cache와 근본적으로 같린다. Transformer의 decode는 “weight 갱신 없음”이 불변식이었다 (KV를 append하고 lookup만). 이 계열은 그 불변식을 **폐기**한다 — decode 안에 gradient의 forward+backward가 들어온다. 대신 상태 크기가 시퀀스 길이 L 과 무관하게 **상수** (W 의 크기)라는 게 존재 이 유다. 모델링 시 “토큰당 backward 1회”를 decode 비용에 반드시 넣어야 한다.

5. 왜 chunk가 병렬화를 가능하게하나

토큰을 하나씩 순서대로 갱신하면 병렬화가 안 된다 (W_t 가 W_{t-1} 에 의존하니까). 그래서 이 계열은 **chunkwise parallel training**을 쓴다. 핵심 트릭: chunk안의 모든 gradient를 **chunk** 시작 시점의 고정된 상태에서 계산한다.

$$W_t = W_{\xi(t,C)} - \sum_{\tau=\xi(t,C)+1}^t \eta_{\tau} \nabla_W \ell(W_{\xi(t,C)}; k_{\tau}, v_{\tau})$$

기호 풀이 - C = chunk 크기 (한 묶음의 토큰 수). - $\xi(t, C)$ = 토큰 t 가 속한 chunk의 시작 위치. 예: $C = 64$ 면 ξ 는 0, 64, 128, ... 중 t 바로 앞의 값. - $W_{\xi(t,C)}$ = chunk 시작 시점에 얼려둔 (frozen) 메모리 상태 = **snapshot**. - $\sum$ = chunk 시작부터 t 까지의 gradient를 전부 더한 것 (누적 합).

직관 chunk안의 모든 토큰이 **같은 출발점** (W_{ξ})에서 gradient를 쏜다. 출발점이 고정이니 이 gradient들은 서로 독립 — 한 번의 큰 batched forward+backward로 몰아서 계산할 수 있다. 각 토큰의 상태는 그냥 gradient들의 누적 합. 남은 순차 의존성은 **chunk 경계 하나뿐**이다 (한 chunk의 마지막 상태가 다음 chunk의 시작이 됨).

이 식은 ~라는 뜻이다. “정확히는 매 토큰마다 상태가 바뀌어야 하지만, chunk안에서는 시작 상태로 근사하자”는 타협. 그래서 chunk가 클수록 근사가 낫고 (stale), 작을수록 신선하지만 작업이 잘게 쪼개진다. §1의 딜레마가 여기서 나온다.

주의 이 chunk 근사는 FlashAttention 같은 tiling과 **다르다**. tiling은 계산 순서만 바꿀 뿐 결과가 bit 단위로 똑같다 (exact). chunk는 **계산되는 함수 자체를 바꾼다** (semantic). 그래서 chunk 크기는 단순한 성능 손잡이가 아니라 모델의 동작을 바꾸는 hyperparameter다 — 이게 §2의 mismatch가 생기는 이유다.

6. Stage 1 — 계층 메모리: global 하나 + local N개

이제 TNT의 핵심 구조다. 한 문장 요약: 큰 chunk로 도는 순차적 global memory 하나가 long-range 맥락을 담당하고, 주기적으로 초기화되는 여러 개의 병렬 local memory가 세밀한 정보를 담당한다.

비유 회사의 기억을 두 종류로 나눈 것이다. **global memory** = 회사전체의 연혁을 두 겹께 정리하는 연간 보고서 (가끔, 크게, 오래 남김). **local memory** = 이번 주 회의 메모 (자주, 작게, 매주 새 노트로 갈아치움). 두 기록을 합쳐서 답한다.

6.1 Global memory — 크고 느리게

Global memory 상태 W^g 는 아주 큰 chunk 크기 C_g (실험에서 2048)로 §5의 표준 chunk recursion을 돈다. 상태는 시퀀스 전체를 관통해 순차적으로 이어지므로 long-range 정보가 살아남는다.

기호 풀이 W^g = global 메모리의 fast weights. C_g = global chunk 크기 (=2048). 16K 길이 시퀀스라면 순차 handoff가 $16384/2048 = 8$ 번뿐.

직관 C_g 가 크니 갱신은 드물지만 한 번에 크고 dense한 행렬 곱이 된다 → 설계상 **compute-bound** (하드웨어를 꽉 찰). §1의 트럭비유에서 “꽉 채운 트럭”이 이것이다. 대신 갱신이 드무니 반응은 느리지만, long-range라 원래 그래도 된다.

6.2 Local memory + periodic reset — 병렬화의 열쇠

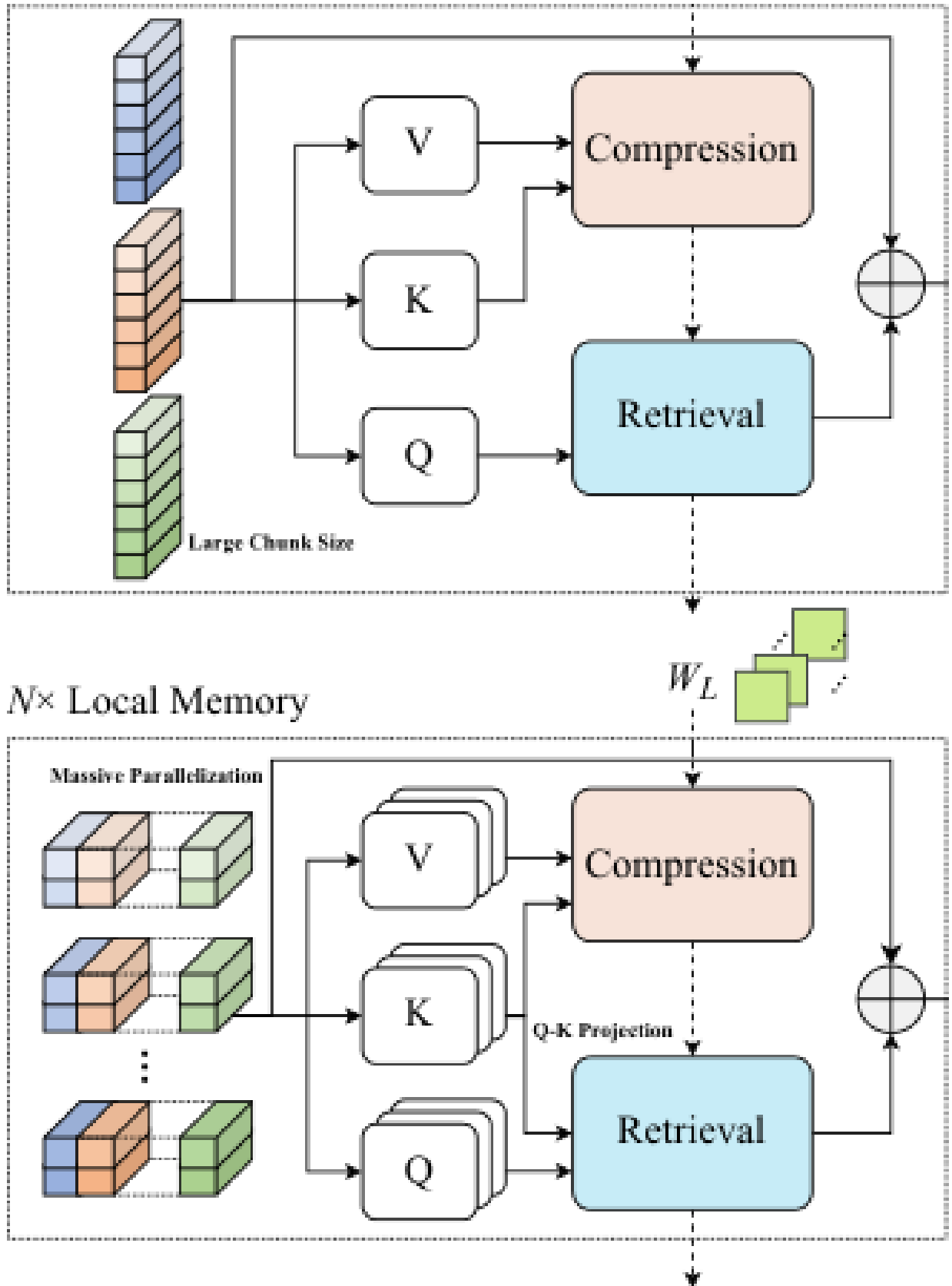
핵심 혁신은 local 쪽에 있다. Local memory는 작은 chunk 크기 C_l 로 돌되, 일정 주기 (L_s 토큰)마다 학습된 초기 상태 W_{init} 으로 리셋된다.

기호 풀이 - W^l = local 메모리의 fast weights. - C_l = local chunk 크기 (작음, 예: 8). - L_s = shard 길이 = 리셋 주기 (예: 2048 또는 4096 토큰). - W_{init} = 학습 가능한 초기 상태. 매 shard가 여기서 시작한다. 0이 아니라 훈련으로 meta-learn된 “좋은 출발점”. - shard = 두 리셋 사이의 구간. 시퀀스는 L/L_s 개의 shard로 쪼개진다.

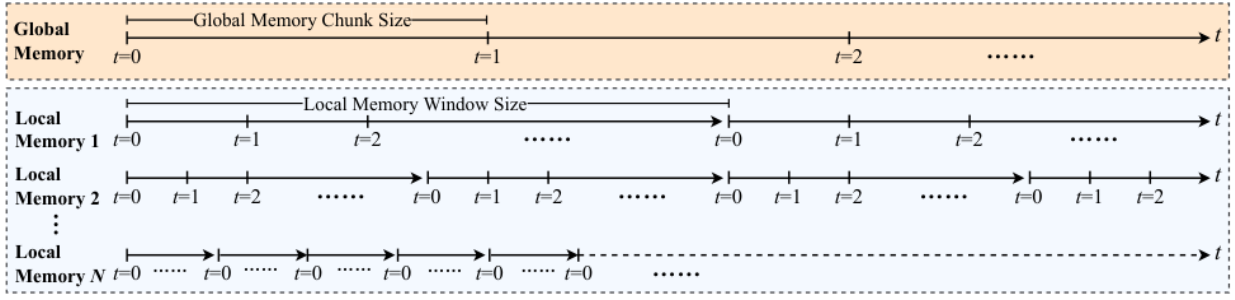
직관 왜 리셋이 결정적인가? 비선형 recurrence는 **parallel scan**으로 병렬화할 수 없다 (scan은 결합법칙이 필요한데 MLP를 통과하는 상태 전이엔 없다). 리셋은 그 병렬화 불가능한 사슬을 **아예 끊어버린다**. 각 shard가 항상 같은 W_{init} 에서 시작하므로, shard들은 서로에게 완전히 독립이다 — 여러 장치에 흩뿌리거나 (context parallelism), 한 accelerator의 batch 축에 쌓아 커널을 살찌울 수 있다.

이 식은 ~라는 뜻이다. “긴 순차 사슬을, 서로 독립인 짧은 조각 여러 개로 잘라 병렬로 돌린다.” §5의 순차 병목을 리셋으로 물리적으로 절단한 것.

한계 공짜가 아니다. local memory는 shard 경계에서 모든 것을 잊는다. 그 손실을 메우는 게 global memory의 존재 이유다 — 리셋 없는 global이 long-range를, 리



TNT Stage 1의 아키텍처 개관. 위 블록: 큰 chunk로 순차적으로도는 하나의 global memory(long-range 담당). 아래 블록: 학습된 초기상태에서 주기적으로 재초기화되어 대량 병렬화(Massive Parallelization)되는 N개의 local memory. 두 메모리 모두 Compression(write)·Retrieval(read) 두 연산을 갖고, 두 경로 출력이합산되어 y_t 가 된다 — 단 Q-K Projection은 local 경로에만 붙는다. 출처: 저자, TNT(arXiv:2511.07343) Fig.3 — 원저자 그림.



TNT 메모리 계층의 시간축 도해. 같은 행에서 같은 t 값의 갱신은 동시에 (병렬로) 실행되고 $t=0$ 은 메모리 초기화를 뜻한다. 맨 위 global memory 는 큰 chunk 하나가 시퀀스 전체를 순차적으로 관통하는 반면, 아래 N 개의 local memory 는 각자의 window(shard) 길이마다 $t=0$ 으로 reset 되어 shard 들이 서로 독립 · 병렬이 된다 — index 가 커질수록 window 가 짧아 더 자주 reset 된다. 출처: 저자, TNT(arXiv:2511.07343) Fig.1 — 원저자 그림.

셋 있는 local 이 병렬성을 든다. 실제로 ablation 에서 global 을 빼면 품질이 21.04 → 25.60 으로 붕괴한다 (base 보다도 나쁨). 두부품은 짝으로만 작동한다.

W_{init} 이 학습된다는 점도 하중을 받는다. 모든 shard 가 0 이 아니라 meta-learn 된 좋은 prior 에서 출발하므로, 리셋이 “정보전멸” 이 아니라 “좋은 출발점으로 복귀” 가 된다. 또 local 은 여러 개일 수있다 — $C_1 = \{4, 8, 16, 32\}$ 처럼 서로 다른 chunk 크기의 local module N 개를 두면 서로 다른 시간 스케일을 잡는다 (module 을 더할수록 품질이 단조 개선).

6.3 Q-K Projection — 읽기를 쓰기의 언어로 되돌리기

작은 문제가 하나 더 있다. 메모리는 “ k (key) 를 넣으면 v (value) 가 나오게” 학습됐는데, **읽을 때는 q (query) 로 읽는다**. 학습된 함수를 그 입력 domain 밖에서 평가하는 셈이라 품질이 깎인다 (일종의 미니어처 train/test mismatch).

TNT의 처방: query 를 그대로 넣지 말고, 지금까지 본 key 들이 만드는 부분공간으로 사영한 뒤 넣는다. 사영 행렬 Π_t 는 key 들의 outer product 를 계속 누적한 것이다:

$$\Pi_t = \Pi_{t-1} + k_t k_t^\top \quad (\text{shard 시작에서 } k_t k_t^\top \text{로 리셋})$$

기호 풀이 Π_t = Q-K projection 행렬 ($d \times d$). $k_t k_t^\top$ = key 벡터의 outer product (rank-1 행렬, $d \times d$). 이것 계속 더해 나간다. local 과 같은 리셋 규칙을 따른다.

직관 $\Pi_t q_t$ 는 “지금까지 본 key 들의 방향으로 query 를 정렬” 시킨다. 자주 본 방향일수록 증폭된다. 과거 key 를 저장할 필요가 전혀 없다 — 상태는 상수 크기 ($d \times d$) 의 running sum 하나뿐.

이 식은 ~라는 뜻이다. attention 이 읽는 순간 $q^\top K$ 로 하던 “query 를 key 통계와 대면시키는 일” 을, 상수 크기 누적 합으로 미리 압축해 두는 것.

최종 읽기는 global 과 local 의 출력을 더한 것이다:

$$y_t = \mathcal{M}(q_t; W^g) + \mathcal{M}(\Pi_t q_t; W^l)$$

두 가지 비대칭에 주목: (1) projection은 **local**에만 붙는다 (세밀한 local이 mismatch에 더 민감하므로; global은 raw query를 받아 계산을 아낀다). (2) global 읽기는 chunk 시작에 열린 상태를 읽는다 (그래서 최대 $C_g - 1$ 토큰 낚은 정보를 읽지만, 그만큼 읽기도 chunk-병렬이된다).

시스템 모델링 관점 Π_t 갱신 ($+k_t k_t^\top$)은 KV cache append의 rank-1 GEMM 대응물이고, $\Pi_t q_t$ 는 mat-vec 한 번 ($\sim 2d^2$ FLOPs). 즉 projection은 시퀀스 길이와 무관한 상수 크기 (d^2 개 숫자) 상태를 하나추가할 뿐, 순차 병목을 만들지 않는다. 모델링 시 local module 당 상태 = fast weights W^l + projection $\Pi(d^2)$ 로 잡으면된다.

7. Stage 2 — 작은 chunk로 짧게 fine-tuning

Stage 1이 훈련 효율을 해결했지만 §2의 절벽 (chunk-size mismatch) 이남는다. 큰 chunk로 훈련한 모델을 그냥 작은 chunk로 평가하면 품질이떨어진다.

TNT의 관찰: **짧은 fine-tuning**으로 이 불일치가 교정되며, 원래성능을 회복하는 정도가 아니라 넘어선다. Stage 2는 pre-train된 모델을 더 작은 **local chunk** 크기 C'_l 로 계속 훈련하는것이다 (global의 C_g 는 그대로). 비용은 pre-training의 약 5~8%(Stage 1이 3~5.5시간일 때 Stage 2는 0.15~0.46시간).

핵심 이상적 목표는 $C'_l = 1$ 이다. 이 지점이 autoregressive 서빙과정확히 맞물린다: **global memory**가 큰 chunk로 **prompt**를 흡수하고 (prefill), Stage 2로 적응된 **local memory**가 생성 중 토큰 하나마다 갱신된다 (decode). 이로써 chunk 크기는 더 이상 하나의 타협값이 아니다— Stage 1에서는 **훈련 throughput 손잡이**, Stage 2에서는 **추론 해상도 손잡이**라는 서로 독립인 두 손잡이가 된다.

이것이 이 권의 레시피 **train-big / serve-small**이다.

8. 결과: 얼마나 빨라지고, 품질은?

8.1 속도 — 목표 품질까지 걸린시간

실전 지표는 “같은 training loss(3.20)에 도달하는 시간”이다 (150M 모델).

모델	chunk	시간 (hrs)	배속
Titans	8	19.48	1.00×
Titans	64	4.18	4.67×

모델	chunk	시간 (hrs)	배속
Titans	128	3.71	5.25×
TNT	{64}	1.12	17.37×
TNT	{128}	1.16	16.75×
TNT	{8}	2.54	7.68×
Gated Transformer (FlashAttention)	-	0.96	20.22×

직관 헤드라인 17.37 배는 “가장 정확한 Titans”(C = 8) 대비다. 중요한 건 **같은 chunk 8 끼리 비교해도 TNT가 7.7 배 빠르다**는 것 — 이득이 단순히 chunk 를 키운 것만이 아니라 **구조 (리셋 병렬화)** 에서 왔다는 증거다.

한계 정직하게: 커널 최적화된 Gated Transformer(0.96h) 는 아직 못 이긴다. TNT 는 순수 JAX 구현이고 fused custom kernel 이 없어서다 — 논문도 이를 future work 로 명시한다. 병렬 구조만으로 여기까지 온 것이고, 커널까지 최적화하면 더 갈 여지가 있다는 뜻.

8.2 품질

모델	구성	평균 ppl ↓	평균 acc ↑
Transformer (gating 없음)	-	23.58	38.3
Gated Transformer	-	22.39	39.7
Titans	C = 8	25.07	39.0
TNT Stage 1	{4,8,16,32}	23.13	40.6
TNT Stage 2	{2,4,8,16}	23.09	40.9

Stage 1 만으로 모든 RNN baseline 과 바닐라 Transformer 를 이기고, Stage 2 가 품질을 더 내린다. reasoning 정확도 (acc) 에서는 Gated Transformer 까지 이긴다. 단 **perplexity** 에서는 **Gated Transformer(22.39) 에 여전히 진다** — 논문이 스스로 명시하는 격차다.

주의 이 모든 검증은 **momentum·gating·Muon** 을 “명료성을 위해” 제거한 단순화 **Titans** 위에서 이뤄졌다. 즉 앞 권 (G03~G05) 의 분류모델과의 결합은 측정된 적이 없다. 그리고 실증 범위가 150M/10B tokens 로이 계열에서 가장 작은 축이다 — “17 배 가속이 1B+에서도 성립하는가” 는 열린 질문.

9. 시스템 모델링 관점 정리

이 권의 가장 중요한 부분이다. TNT 를 시스템 모델에 넣을 때 잡아야 할 좌표들을 모은다.

시스템 모델링 관점 — 병렬 구조. 훈련/추론의 계산 그래프는 세 조각이다. - **global:** 순차 깊이 L/C_g (16K 에서 8 번), 각 handoff 가 2048-토큰 batched

matmul $\rightarrow$ compute-bound. - **local**: 완전히 독립인 L/L_s 개 shard $\rightarrow$ 장치 간상태 교환 0의 진짜 context parallelism, 또는 batch 축에 쌓아 커널을살찌움. - Π (**projection**): $d \times d$ carry 하나의 prefix scan $\rightarrow$ 순차 병목 없음.

즉 TNT는 “시퀀스 길이만큼 긴 단일 파이프라인”을 **굵은 파이프하나 (global)** + **독립적인 짧은 파이프 묶음 (local)** + **scan 커널하나 (Π)**로 재배관해 arithmetic intensity를 제조한다.

시스템 모델링 관점 — 상태 크기 vs KV cache. decode 시점, 메모리 layer 당 상태 = $(1+N)P_f + Nd^2$ 개 값. - P_f = 메모리 sub-network 하나의 파라미터 수. 표준 deep memory(expansion 4, 2-layer MLP)면 $P_f \approx 8d^2$. - 예: $N = 4$ local 이면 상태 $\approx 5 \cdot 8d^2 + 4d^2 = 44d^2$. $d = 1024$ 면 약 46M 값 = bf16으로 $\sim 92\text{MB/layer}$. **가볍지 않다.** - 하지만 KV cache($2Ld/\text{layer}$)가 이를 넘는 지점은 대략 $L \gtrsim 22d = d = 1024$ 면 **약 22K 토큰**. 그 뒤로 KV cache는 무한히 자라고 TNT 상태는 그대로 상수. 이게 긴 문맥에서의 존재 이유.

시스템 모델링 관점 — decode traffic & placement. $C'_1 = 1$ decode는 토큰마다 local fast weights P_f 개값의 read-modify-write(읽고 $\rightarrow$ forward+backward $\sim 6P_f$ FLOPs $\rightarrow$ 쓰기)다. 값당 FLOP이 한 자릿수라 **memory-bound**(KV cache 재독과같은 체질이되 트래픽이 L 에 무관·상수). 그리고 갱신 주기가배치 위치를 결정한다: 매 토큰 갱신되는 $W^1 \cdot \Pi$ 는 on-chip에 상주시킬 대상, 2048 토큰에 한 번 크게 갱신되는 W^8 는 HBM에 두고 드물게 대량으로 만질 대상.

시스템 모델링 관점 — 운영. per-request fast-weight 상태는 shared-weight batching을 깨므로, local 경로 batching은 request 별 weight를 갖는 grouped-GEMM 형태. 뜻밖의 선물: 리셋 덕분에 **preemption/restore** 때 복구할 **local** 상태 **이력이 최대 L_s 토큰으로 유계**다 (shard 시작 상태는 언제나 상수 W_{init}). 다중 해상도 ($N \uparrow$)는 decode 비용을 N 에비례해 늘리고, 품질 대가는 module 당 대략 -0.3 ppl.

주의 위 prefill/decode 서사는 **아키텍처 논증이지 벤치마크가 아니다**. chunk-1 모드의 decode throughput/latency를 KV-cache Transformer와 맞대잔 실측은 TNT 논문에 없다 (이 계열 6편 전체에 decode wall-clock 수치가부재).

10. 한계

- **단순화 위의 검증.** momentum·forget gating·Muon inner optimizer를 모두 뺀 plain-GD 메모리로 검증. 완전한 Titans/Atlas에 17배가속과 Stage 2가 이식되는지는 미측정.
- **mismatch의 증거 폭.** chunk-size mismatch는 단일 설정 (550M, 그림 하나)이 근거. 왜 생기는지, Stage 2가 무엇을 고치는지에대한 이론이 없다.
- **리셋 아래의 recall.** local은 L_s 마다 전부 잊고 global은 낡은 frozen 상태를 읽는다. shard 경계를 넘는 정밀 recall(needle-in-a-haystack 류)이 얼마나 살아남는지 미평가 — 이 보험의실효성이 가장 큰 빈칸이다.

- **스케일 · 커널**. 150M/10B tokens 범위, Gated Transformer 대비 남은 ppl 격차 (22.39 vs 23.09), custom kernel 부재. 논문은 이 셋을 감추지 않고 명시한다 — 이 계열에서 드물게 systems 논문다운 정직성이다.

요약 - 문제: chunk 크기 하나가 훈련 속도 (크면 빠름)와 추론품질 (작으면 좋음)을 동시에 결정해 딜레마. 게다가 모델이 훈련 chunk에 과적응해 서빙 때 chunk를 바꾸면 품질이 무너진다 (chunk-size mismatch). - **해법:** (1) **계층 메모리** — 큰 chunk global(long-range-compute-bound) + 작은 chunk local N개 (세밀 · 병렬). (2) **periodic reset-to- W_{init}** — 비선형 recurrence의 순차 사슬을 끊어 shard를 완전히 독립 · 병렬화 (context parallelism). (3) **Q-K projection** — 읽기를 쓰기 domain으로 정렬 (d^2 상수상태). (4) **two-stage training** — Stage 1은 큰 chunk로 최대 throughput pre-train, Stage 2는 58% 비용으로 작은 chunk fine-tune → chunk-1 decode를 품질과 정렬 (train-big / serve-small). - **성과:** 150M에서 가장 정확한 Titans 대비 최대 17.37배 빠르면서 품질도 개선. 단 커널 최적화 Gated Transformer는 아직 못 이기고, 검증은 단순화 모델 · 작은 스케일에 한정. - **시스템 렌즈:** 이 권은 “표현력”이 아니라 **arithmetic intensity(FLOPs 활용률)의 제조**에 관한 이야기다. chunk 크기 = 작업 단위의 intensity, reset = 순차 축을 batch 축으로 되돌리는 절단.

다음 권 예고. TNT는 이미 세 개 층위의 update frequency(global 2048 토큰마다 · local 매 토큰 · slow weights 훈련에서만)로 도는 시스템이지만 그것을 “공학적 방편”으로만 썼다 — 다음 권 **G07 · Nested Learning**은 그 관찰을 뒤집어, 모델과 훈련 절차 전체가 각자의 update frequency로 도는 중첩된 optimization 문제들의 시스템이라는 존재론으로 승격시킨다.